ComplianceIA — Plataforma de Evaluación de Gobernanza de IA

> *Proyecto Capstone para **Therion Labs**. Autodiagnóstico de madurez en gobernanza de Inteligencia Artificial para empresas, con validación de evidencias asistida por LLM, reportes de madurez, rutas formativas y panel de estadísticas comparativas por rubro.*

Cliente	Therion Labs (Seguridad y Gobernanza de IA, miembro de la Comisión de Seguridad IA – CCHIA)
Marcos normativos	Ley N.º 21.719 (Chile), ISO/IEC 42001, NIST AI RMF 100-1, NIST AI 600-1, OWASP Top 10 for LLM
Metodología	Scrumban (Por hacer → En curso → Listo, con límite WIP)
Meta de madurez	TRL 5 — validación de componentes en entorno representativo (local, simulando uso real)
Stack	FastAPI · SQLAlchemy 2.0 async · PostgreSQL 16 · Redis 7 · ARQ · LangChain + Groq · spaCy · Docker Compose · nginx · HTML/JS estático
Última actualización	15-oct-2026 (T-04: optimización del motor NER)

Tabla de contenidos

1. Contexto del proyecto
2. Estado actual
3. Arquitectura
4. Estructura del repositorio
5. Flujos funcionales
6. Base de datos
7. Endpoints de la API
8. Capa de Políticas e IA
9. Seguridad y privacidad (Ley 21.719)
10. Frontend
11. Datos maestros
12. Puesta en marcha
13. Variables de entorno
14. Pruebas y calidad
15. Comandos útiles y solución de problemas
16. Backlog, deuda técnica y riesgos

1. Contexto del proyecto

Problema

La IA corporativa se está adoptando sin políticas de seguridad, las auditorías se hacen a mano en hojas de cálculo y la entrada en vigencia de la **Ley 21.719** (protección de datos personales) presiona a las organizaciones a demostrar cumplimiento. Como resume el cliente: *la IA no falla por falta de tecnología, sino por falta de capacidad interna para gobernarla.*

Solución: tres funciones principales

Función	Qué entrega
Autodiagnóstico	Cuestionario con filtro de aplicabilidad (7 preguntas) y hasta 40 controles; las evidencias se validan con un LLM; produce un reporte de madurez y de brechas.
Rutas formativas	Formación en 3 niveles (Inicial, Intermedio, Avanzado) × 4 dominios, asignada según las brechas detectadas.
Panel de estadísticas	Benchmark por rubro y dominio (promedio, mediana, máx., mín.) sin exponer datos de ninguna organización.

Equipo

Integrante	Rol
Jerson Reyes	Product Owner
Jason Sánchez	Scrum Master
Ángel Muñoz	Developer Lead
Rodrigo Álvarez	Docente guía

Épicas y sprints

Épicas: (1) Motor de Autodiagnóstico y Motor de Reglas · (2) Procesamiento Inteligente y Privacidad de Datos con IA · (3) Rutas formativas e Interfaz Dashboard.

Sprint	Objetivo
Sprint 1	Desarrollo e implementación del backend y reglas de cálculo
Sprint 2	Desplegar motor NER, gestión de derechos ARCOP y cifrado
Sprint 3	Implementar IA, controles, detección de brechas y benchmarking
Sprint 4	Despliegue del frontend, automatización del pipeline y staging para pruebas

2. Estado actual

Área	Estado	Detalle
Backend FastAPI (Clean Architecture)	☐ Hecho	11 routers, 35 endpoints
Base de datos y migraciones	☐ Hecho	19 tablas, 13 migraciones Alembic, seeds de catálogos, controles y filtro
Filtro de aplicabilidad (F1–F7)	☐ Hecho	Árbol de decisión aditivo (22 controles base + 18 condicionales)
Respuestas y reglas de clasificación	☐ Hecho	Matriz de 5 estados declarables (HU-02): `requiere_revision` la elige el usuario y la resuelve el LLM; justificación obligatoria en parcial y revisión
Validación de evidencias con LLM (worker ARQ + Groq)	☐ Hecho	Asíncrona; el backend decide, el LLM solo propone
Capa de Políticas e IA (gateway, prompts versionados, métricas)	☐ Hecho	`GET /api/ia/metricas`, revisión humana
Motor NER de anonimización + cifrado del mapa de entidades	☐ Hecho	spaCy `es_core_news_md` + reglas; fail-closed. **v1.1.0 optimizada (T-04): 25 ms por página (p95), precisión 0,988, cobertura de PII 100 %**
PII → tokens antes de persistir (HU-06)	☐ Hecho	Justificaciones y nombres de archivo anonimizados al guardar; evidencias cifradas en disco y reemplazadas por su texto anonimizado; mapa de entidades irreversible; textos ARCOP cifrados (excepción); prueba de regresión sobre BD, disco y logs. Ver `backend/CAMBIOS_HU06.md`
Capa anti prompt-injection	☐ Hecho	Bloqueo preventivo y bandeja de eventos
Reportes: cierre, resumen por dominio, comparativa, benchmark	☐ Hecho	Benchmark recalculado por cron diario (ver deuda técnica #4)
Derechos ARCOP (Ley 21.719) + autenticación por roles	☐ Hecho	Titular y administrador; anonimización al aprobar una supresión
Dockerización completa	☐ Hecho	`docker compose up -d --build` levanta los 6 servicios
Suite de pruebas (pytest)	☐ Hecho	133 funciones de prueba (212 casos) + prueba en vivo, prueba de latencia y pruebas de rendimiento del NER
Frontend	🟡 Demo estática	14 páginas HTML funcionando con una API **simulada** en `localStorage`; falta conectarlo al backend
Alta de invitaciones (usuario temporal) y	☐ Pendiente	Hoy se crean por SQL

Área	Estado	Detalle
evaluaciones por API		
Autenticación en todos los endpoints	☐ Pendiente	Solo `/api/arcop` y `/api/auth/me` exigen token
CRUD de administración de matriz/árbol (backend)	☐ Pendiente	Existe solo en la demo del frontend
Pipeline CI/CD y staging	☐ Pendiente	Sprint 4

El detalle priorizado de lo pendiente está en la sección 16.

---

3. Arquitectura

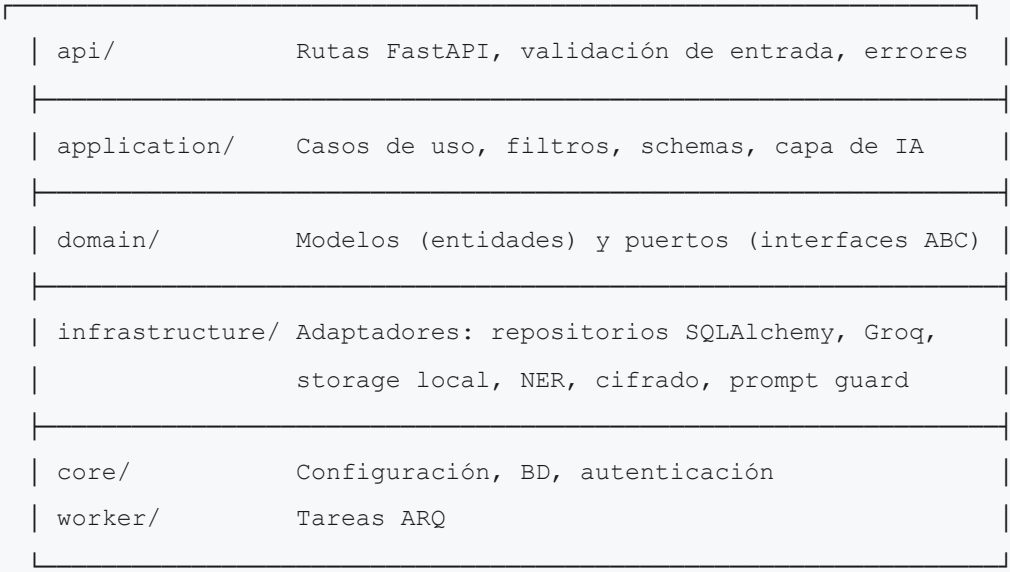
Vista de componentes

```
flowchart LR
      U[Usuario<br/>navegador] -->|:8080| N[frontend<br/>nginx 1.27]
      N -->|HTML/JS estático| U
      N -->|/api/* proxy| A[api<br/>FastAPI + uvicorn :8000]
      A -->|SQL async| DB[(db<br/>PostgreSQL 16)]
      A -->|encola job| R[(redis<br/>Redis 7)]
      A -->|guarda archivo| S[[volumen storage]]
      W[worker<br/>ARQ] -->|toma job| R
      W -->|lee archivo| S
      W -->|SQL async| DB
      W -->|texto ANONIMIZADO<br/>HTTPS| G[Groq LLM]
      M[migrate<br/>alembic upgrade head] --> DB
```

Servicio Docker	Imagen	Puerto (solo `127.0.0.1`)	Función
`frontend`	`nginx:1.27-alpine`	8080	Sirve `frontend/` y reenvía `/api` y `/health` al backend (mismo origen, sin CORS). Límite de subida 20 MB
`api`	`complianceai-backend`	8000	FastAPI (`/docs` para Swagger)
`worker`	`complianceai-backend`	—	ARQ: valida evidencias y recalcula el benchmark (cron 02:00 UTC)
`migrate`	`complianceai-backend`	—	`alembic upgrade head` y termina
`db`	`postgres:16`	5433	Base de datos `complaince_db`
`redis`	`redis:7-alpine`	6379	Cola de trabajos de ARQ

Orden de arranque: `db` sano → `migrate` termina bien → `api` y `worker` → `frontend`. La API y el worker comparten el volumen `storage` (evidencias). La imagen del backend es única (multi-stage, Python 3.13-slim, usuario sin privilegios) e incluye spaCy con el modelo `es_core_news_md`.

Clean Architecture



Regla de dependencia: las capas externas dependen de las internas, nunca al revés. Cambiar el LLM (Groq → otro proveedor) o el storage (local → S3) es reemplazar un adaptador sin tocar la lógica de negocio. Puertos definidos: `LLMProviderPort`, `SanitizerPort`, `RegistroIAPort`, `PromptGuardPort`, `EvidenceValidatorPort`, `FileStoragePort`.

Principios de diseño

1. **El LLM nunca accede a la BD.** Solo recibe texto (prompt) y devuelve JSON validado con Pydantic. El backend es el único que lee y escribe.
2. **El backend decide, el LLM propone.** Umbral de confianza, degradación y asignación de brechas son reglas de negocio del backend.
3. **Ningún dato personal sale en claro.** Todo texto no confiable pasa por el motor NER antes de llegar al proveedor; si la sanitización falla, se aborta (*fail-closed*).
4. **Nada se guarda con PII en claro (HU-06).** Justificaciones y nombres de archivo se anonimizan antes de escribirse; los archivos de evidencia viven cifrados hasta que el worker los reemplaza por su texto anonimizado; los textos ARCOP se cifran (excepción documentada).
5. **Todo queda auditado.** Cada llamada al LLM se registra en `llamadas_llm`; cada mapa de entidades (tipos, huellas HMAC y conteos, sin valores: HU-06) en `sanitizaciones`; cada intento de inyección en `eventos_seguridad`; cada cambio ARCOP en `eventos_solicitud_arcop`.

---

4. Estructura del repositorio

```
Capstone/
  ├── Documentos/                      # Entregables académicos por fase e integrante
  │   ├── Fase 1/                      # Guía, formativa, bitácora CPCITT, presentación,
  autoevaluaciones
  │   └── Fase 2/                      # Diarios de reflexión
  ├── Extras/
  │   └── Guia_ComplianceAI_PromptInjection.docx.pdf
  └── Proyecto/
      ├── README.md                    # ← este documento
      ├── docker-compose.yml           # Sistema completo (6 servicios)
      ├── docker/nginx.conf            # Frontend estático + proxy /api
      ├── frontend/                    # Demo estática (ver sección 10)
      │   ├── *.html                   # 14 páginas
      │   ├── css/styles.css
      │   └── js/                      # data.js, logica.js, store.js, ui.js, pdf-inform
  e.js
      └── backend/
          ├── Dockerfile               # Imagen única para migrate, api y worker
          ├── requirements.txt
          ├── .env.example             # Plantilla de configuración (copiar a .env)
          ├── env.example              # Variante con valores para Docker local (ver deu
  da #12)
          ├── alembic.ini
          ├── CAMBIOS_ARCOP.md         # Detalle funcional de la historia de usuario ARC
  OP
          ├── CAMBIOS_T04.md           # T-04: optimización del motor NER (resultados y
  decisiones)
          ├── reporte_latencia_live_c10.md
          ├── evidencia.txt / evidencia_maliciosa.txt   # Archivos de ejemplo para prue
  bas manuales
          ├── migrations/versions/     # 13 migraciones Alembic
          ├── scripts/benchmark_ner.py  # Benchmark y profiling del motor NER (T-04)
          ├── tests/                   # Suite pytest (ver sección 14)
          └── app/
              ├── main.py              # App FastAPI, routers y pool ARQ
              ├── core/
              │   ├── config.py        # Settings (pydantic-settings) + validaciones de
  producción
              │   ├── database.py      # Engine async, sesión, Base
              │   ├── autenticacion.py# Tokens HMAC, roles, hash de contraseñas, depend
```

```
encias
            │   └── crear_admin.py  # CLI para crear/desactivar administradores
            ├── api/routes/          # arcop, auth, catalogos, controles, evaluacione
s, evidencias,
            │                        # filtro, metricas_ia, reportes, respuestas, segu
ridad
            ├── application/
            │   ├── ai/              # gateway.py, politicas.py, metricas.py, evidence
_validator.py,
            │   │                    # prompts/ (base.py, validar_evidencia.py)
            │   ├── filters/control_tree.py   # Árbol de aplicabilidad F1-F7
            │   ├── schemas/         # api_schemas, arcop_schemas, evidence_validation
            │   └── use_cases/       # create_respuesta, upload_evidence, process_evid
ence,
            │                        # responder_filtro, generate_resumen, update_benc
hmark,
            │                        # obtener_metricas_ia, registrar_evento_segurida
d,
            │                        # gestionar_solicitud_arcop, iniciar_sesion
            ├── domain/
            │   ├── models/          # 19 entidades SQLAlchemy (ver sección 6)
            │   └── ports/           # Interfaces ABC
            ├── infrastructure/
            │   ├── llm/             # groq_provider.py, factory.py
            │   ├── repositories/    # 10 repositorios
            │   ├── security/        # ner_sanitizer.py, cifrado.py, prompt_guard.py,
seguridad.py
            │   └── storage/         # local_storage.py, archivo_cifrado_storage.py (H
U-06)
            └── worker/tasks.py      # process_evidence_task + cron update_benchmark_t
ask
```

5. Flujos funcionales

5.1 Recorrido completo de un diagnóstico

```
flowchart TD
      A[Invitación: usuario_temporal + rubro] --> B[Crear evaluación]
      B --> C[Filtro de aplicabilidad<br/>POST /evaluaciones/id/filtro]
      C -->|F1 = NO| Z[Ningún control aplica]
      C -->|F1 = SI| D[Controles aplicables<br/>22 base + condicionales]
      D --> E[Responder cada control<br/>POST /respuestas]
      E -->|cumplido / parcial| F[Subir evidencia<br/>POST /respuestas/id/evidencias]
      E -->|no_cumplido| G[Brecha asignada al instante]
      E -->|no_aplica| H[Sin brecha, fuera del puntaje]
      F --> I[Worker: seguridad → NER → LLM → reglas]
      I --> J[Clasificación final]
      G --> K
      H --> K
      J --> K[Cerrar evaluación<br/>POST /evaluaciones/id/cerrar]
      K --> L[Resumen por dominio + puntaje general + nivel]
      L --> M[Comparativa vs. benchmark del rubro]
      L --> N[Brechas → rutas formativas]
      O[Oficial de Cumplimiento] -.->|PATCH /respuestas/id/revision| J
```

> *Los pasos A y B aún no tienen endpoint: se crean por SQL (ver 12.3).*

5.2 Filtro de aplicabilidad (preguntas de compuerta)

Las 7 preguntas se responden SI/NO y deben venir **todas** en un mismo envío (se puede rehacer: las anteriores se reemplazan). F1 es la compuerta raíz; F2–F7 suman controles al conjunto base.

Código	Pregunta	Controles que activa (si SI)
F1	¿La organización utiliza, desarrolla o integra algún sistema, modelo o herramienta de IA?	**Base (22):** DAT-001, 003, 004, 005, 006, 010 · GOB-001, 002, 003, 004, 005, 007, 009, 010 · RIE-001, 003, 004, 008, 010 · SEG-006, 007, 009. Si es NO no aplica ninguno
F2	¿Procesa datos sensibles (salud, biométricos o perfilamiento de personas/menores)?	DAT-002, DAT-009
F3	¿Usa servicios, APIs, SaaS o plataformas de IA de terceros o en nube internacional?	DAT-007, DAT-008, GOB-008, SEG-008
F4	¿Usa LLM, IA generativa o arquitecturas RAG / bases vectoriales?	RIE-002, SEG-001, SEG-002, SEG-005
F5	¿La IA genera código/SQL/HTML o ejecuta acciones/agentes conectados a otros sistemas?	RIE-009, SEG-003, SEG-004
F6	¿La IA toma decisiones que afectan derechos, personas, crédito o contrataciones?	RIE-005, RIE-006, SEG-010
F7	¿La empresa desarrolla, entrena o adapta internamente modelos o agentes de IA?	RIE-007, GOB-006

Lógica en `app/application/filters/control_tree.py`.

5.3 Respuesta a un control

Matriz de **5 estados** que el usuario elige (HU-02):

Clasificación declarada	Valor	¿Justificación obligatoria?	¿Lleva evidencia?	¿Pasa por el LLM?	Brecha
Cumplido	`cumplido`	No	Sí	Sí	Solo si el LLM la degrada a `no_cumplido`
Parcialmente cumplido	`parcialmente_cumplido`	**Sí** (mín. 20 caracteres)	Sí	Sí	Solo si el LLM la degrada a `no_cumplido`
No cumplido	`no_cumplido`	No	No	No	**Se asigna al crear la respuesta**
No aplica	`no_aplica`	No	No (subirla da 400)	No	No
Requiere revisión	`requiere_revision`	**Sí** (mín. 20 caracteres)	**Sí, obligatoria para completar**	Sí: **el LLM decide** `cumplido`, `parcialmente_cumplido` o `no_cumplido`	Si el LLM decide `no_cumplido`

- **Requiere revisión** (decisión del PO, 16-oct): el usuario la elige cuando no sabe cómo clasificar el control. Adjunta evidencia y el LLM la clasifica (prompt `validar_evidencia/1.2.0`). Si el LLM no alcanza el umbral de confianza, el control queda en `requiere_revision` para la **revisión humana** (`PATCH /revision`). Mientras no se resuelva puntúa 0. La capa de seguridad también puede asignar este estado (`fuente = seguridad`).
- **Justificación obligatoria** en `parcialmente_cumplido` y `requiere_revision`: al menos `JUSTIFICACION_MIN_CARACTERES` (20) caracteres, sin contar espacios repetidos ni de los extremos, y con letras (no solo signos o números). Si no, `422 JUSTIFICACION_OBLIGATORIA` en `POST` y `PUT`.

Al crear la respuesta, la **justificación** pasa por el detector de prompt injection: si es sospechosa se rechaza con `422 PROMPT_INJECTION` y se registra un evento de seguridad.
Reglas del cuestionario (HU-01):

- **Una respuesta por control y evaluación** (restricción única `uq_respuesta_evaluacion_control`). `POST /api/respuestas/` es un *upsert*: la primera vez crea (201) y luego actualiza la misma fila (200, mismo `id_respuesta`). Si se reenvía exactamente la misma declaración no se modifica nada (el guardado automático no borra la validación del LLM); si cambia, la declaración nueva pasa a ser el estado vigente (`fuente = usuario`) y se descarta la validación anterior (LLM o revisión). Las evidencias ya subidas se conservan.
- Solo se aceptan respuestas a evaluaciones **existentes** (404 `EVALUACION_NO_ENCONTRADA`), **abiertas** (409 `EVALUACION_CERRADA` si `estado_progreso` es `completada` o `cerrada`), **con el filtro completado** (409 `FILTRO_PENDIENTE`) y a controles **aplicables** según F1–F7 (422 `CONTROL_NO_APLICABLE`; 404 `CONTROL_NO_ENCONTRADO`). Una evaluación cerrada tampoco permite rehacer el filtro ni subir evidencias. Se cierra con `POST /api/evaluaciones/{id}/completar`.

- `GET /api/evaluaciones/{id}/progreso` devuelve respondidos / aplicables, el avance por dimensión, el siguiente control pendiente y el estado declarado de cada control ya respondido, para **reanudar** el cuestionario. Si se rehace el filtro, las respuestas de controles que dejan de aplicar no cuentan (`respuestas_fuera_de_alcance`).

5.4 Validación asíncrona de evidencias

```
sequenceDiagram
      autonumber
      participant C as Cliente
      participant API as API FastAPI
      participant FS as Storage
      participant Q as Redis (ARQ)
      participant W as Worker
      participant G as Capa IA (AIGateway)
      participant L as Groq
      participant DB as PostgreSQL

      C->>API: POST /api/respuestas/{id}/evidencias (archivo)
      API->>DB: valida respuesta (solo cumplido/parcial)
      API->>FS: guarda archivo (uuid_nombre) + hash SHA-256
      API->>DB: evidencia estado=pending
      API->>Q: enqueue process_evidence_task
      API-->>C: 202 Accepted
      Q->>W: job
      W->>DB: evidencia → processing; carga respuesta, control y brecha
      W->>W: prompt guard sobre justificación y evidencia
      alt inyección detectada (modo bloquear)
          W->>DB: evidencia rejected, respuesta requiere_revision, evento_seguridad
      else texto limpio
          W->>G: ejecutar(VALIDAR_EVIDENCIA, textos, datos)
          G->>G: NER anonimiza (falla ⇒ aborta)
          G->>DB: mapa de entidades CIFRADO (sanitizaciones)
          G->>L: prompt versionado SOLO con texto anonimizado
          L-->>G: JSON (clasificación, confianza, justificación…)
          G->>DB: métrica (llamadas_llm)
          G-->>W: resultado validado
          W->>W: reglas de decisión del backend
          W->>DB: evidencia + clasificación final de la respuesta
      end
```

Reglas de decisión (`ProcessEvidenceUseCase._apply_classification_decision`), con `UMBRAL_CONFIANZA = 0.8` por defecto:

#	Condición	Resultado en la respuesta	Evidencia	Fuente
0	Evidencia marcada como sospechosa y confianza < umbral	`requiere_revision`	`rejected`	`seguridad`
1	Confianza < umbral	Se mantiene lo declarado por el usuario	`rejected`	`usuario`
2	Confianza ≥ umbral	Clasificación propuesta por el LLM	`validated`	`llm`
3	Clasificación final = `no_cumplido`	Se asigna la brecha del control; con cualquier otro estado final se **quita** (`Respuesta.sincronizar_brecha`)	—	—
—	Error de la capa de IA (NER, timeout, rate limit, salida inválida…)	Sin cambios	`error` con mensaje sin PII	—

Estados de una evidencia: `pending` → `processing` → `validated` | `rejected` | `error`. El worker reintenta hasta 3 veces y cada job tiene 300 s de límite.

5.5 Revisión humana

`PATCH /api/respuestas/{id}/revision` permite al Oficial de Cumplimiento fijar la clasificación final (`fuente_clasificacion = revision`). Si queda en `no_cumplido` se asigna la brecha. Esa revisión es la **verdad de referencia** para medir la precisión del LLM en `/api/ia/metricas`.

5.6 Cierre, puntaje y nivel de madurez

`POST /api/evaluaciones/{id}/cerrar` (re)genera la tabla `resumen`:

1. Considera solo controles **aplicables** según el filtro y con respuesta registrada.
2. Excluye `no_aplica`.
3. Puntúa: `cumplido` = 1 · `parcialmente_cumplido` = 0,5 · `no_cumplido` = 0 · `requiere_revision` = 0 (cuenta en el denominador, para que inyectar texto nunca suba el puntaje).
4. Puntaje del dominio = promedio × 100. Puntaje general = promedio ponderado por cantidad de controles.

Puntaje	Nivel de madurez
0 – 25	`inicial`
> 25 – 50	`en_desarrollo`
> 50 – 75	`gestionado`
> 75 – 100	`optimizado`

Si todos los controles quedan fuera (F1 = NO o 100 % `no_aplica`), no se genera resumen y `GET /resumen` responde 404.

5.7 Benchmark sectorial

Tarea cron de ARQ `update_benchmark_task` (diaria, 02:00 UTC): borra y recalcula `benchmark` agrupando los resúmenes por **rubro × dominio** (promedio, mediana, máximo, mínimo, cantidad) de las evaluaciones con `estado_progreso = 'completada'`. `GET /api/evaluaciones/{id}/comparativa` compara la evaluación con el benchmark de su rubro (diferencia contra el promedio).

5.8 Solicitudes de derechos ARCOP (Ley 21.719)

Historia de usuario: *como Titular de Datos Personales quiero registrar solicitudes formales de Acceso, Rectificación, Supresión, Oposición, Portabilidad o Bloqueo*. Tipos: `acceso`, `rectificacion`, `supresion`, `oposicion`, `portabilidad`, `bloqueo`.

```
stateDiagram-v2
      [*] --> recibida: titular crea
      recibida --> en_revision: admin
      recibida --> rechazada: admin
      recibida --> cancelada: titular
      en_revision --> en_proceso: admin
      en_revision --> pendiente_informacion: admin (comentario obligatorio)
      en_revision --> rechazada: admin
      en_revision --> cancelada: titular
      pendiente_informacion --> en_revision: titular aporta información
      pendiente_informacion --> rechazada: admin
      pendiente_informacion --> cancelada: titular
      en_proceso --> resuelta: admin
      en_proceso --> rechazada: admin
      resuelta --> [*]
      rechazada --> [*]
      cancelada --> [*]
```

- Cada cambio queda en `eventos_solicitud_arcop` en la misma transacción.
- Una **supresión resuelta** anonimiza de forma irreversible los datos del titular (email, justificaciones, evidencias y sus archivos, mapas de entidades, extractos de seguridad, descripciones ARCOP) y conserva clasificaciones y puntajes sin vínculo personal. El titular pierde el acceso al instante.
- Cada solicitud tiene un **código de seguimiento único** (`ARC-XXXX-XXXX`); el estado se consulta sin login en `GET /api/arcop/seguimiento/{codigo}`, sin datos personales.
- **Plazos legales por defecto (Ley 21.719):** respuesta en 30 días corridos, prórroga única de 30 días con motivo, bloqueo temporal en 2 días hábiles (configurables con `ARCOP_PLAZO_*`). Se informa `plazo_vencido` y el filtro `?vencidas=true`.
- El titular puede ser un **invitado** o un **encuestado anónimo** de la encuesta pública (entra con la clave de su evaluación).
- Detalle completo, tabla de anonimización y decisiones pendientes: `backend/CAMBIOS_ARCOP.md`.

5.9 Autenticación y roles

Rol	Quién	Cómo obtiene el token
`referente`	Usuario temporal (titular de sus datos)	`POST /api/auth/titular` con `{usuario_temporal_id, email}`; requiere invitación vigente y no anonimizada
`encuestado`	Encuestado anónimo de la encuesta pública (titular de sus datos)	`POST /api/auth/encuestado` con `{evaluacion_id, clave}`; la clave se genera por evaluación (`POST /api/auth/encuestado/clave`, admin; luego HU-19)
`admin_therion`	Administrador de Therion Labs (tabla `usuarios_admin`)	`POST /api/auth/login` con `{email, password}`

- Token bearer firmado con HMAC-SHA256 (`AUTH_SECRET`), vigencia `AUTH_TOKEN_TTL_MIN` (480 min por defecto). Respuesta: `{access_token, token_type, rol, usuario_id, expira_en_min}`.
- Contraseñas con PBKDF2-SHA256 (600 000 iteraciones); verificación en tiempo constante y hash señuelo para no revelar emails existentes.
- En cada petición se comprueba en BD que el admin siga activo o que el titular no esté anonimizado (401 `USUARIO_DESHABILITADO`).
- Administradores: `docker compose exec api python -m app.core.crear_admin --email admin@therionlabs.cl` (contraseña mínima de 12 caracteres; `--desactivar` corta el acceso).
- Token de prueba solo para desarrollo: `python -m app.core.autenticacion --usuario 1 --rol referente`.

---

6. Base de datos

PostgreSQL 16, **19 tablas** + `alembic_version`, **27 claves foráneas**. Los nombres de algunas columnas FK están en mayúscula inicial (`"Rubro_id"`, `"Evaluacion_id"`…): en `psql` hay que escribirlas entre comillas dobles.

6.1 Tablas por capa

Catálogos (datos maestros, sembrados por migración)

Tabla	Filas	Descripción
`dominio`	4	Dominios temáticos
`rubro`	8	Sectores económicos
`ruta_formativa`	12	4 dominios × 3 niveles
`catalogo_brechas`	40	Brechas `ERR-XXX-NNN`: comportamiento detectado, criticidad, acción del sistema y ruta formativa
`matriz_controles`	40	Controles `DAT/GOB/RIE/SEG-NNN`: marco, artículo, obligación, pregunta, evidencia esperada, criticidad, recomendación, test asociado, dominio y brecha
`preguntas_filtro`	7	Preguntas de compuerta F1–F7

Operacionales

Tabla	Descripción
`usuario_temporal`	Invitado por link, con expiración, rubro y `fecha_anonimizacion`
`evaluacion`	Sesión de diagnóstico: estado, `filtro_completado`, fechas
`respuestas_filtro`	Respuesta SI/NO a cada pregunta de compuerta (única por evaluación y pregunta)
`respuestas`	Respuesta a un control (única por evaluación y control): clasificación declarada, justificación, clasificación final, fuente (`usuario` / `llm` / `seguridad` / `revision`), confianza, brecha, revisión humana
`evidencias`	Archivo de respaldo: ruta, MIME, hash, estado de validación, resultado del LLM

Analíticas

Tabla	Descripción
`resumen`	Puntaje y nivel por evaluación × dominio
`benchmark`	Promedio, mediana, máximo, mínimo y cantidad por rubro × dominio

Capa de IA y seguridad (auditoría)

Tabla	Descripción
`llamadas_llm`	Una fila por ejecución de la capa de IA: tarea, proveedor, modelo, versión de prompt, éxito/tipo de error, tokens, costo, latencia, entidades anonimizadas, clasificación y confianza. **Nunca guarda texto del usuario**
`sanitizaciones`	Mapa de entidades por texto: metadatos sin PII (`entidades` con huella HMAC) y conteos. `mapa_cifrado` queda en NULL (HU-06: irreversible; solo se llena con `GUARDAR_MAPA_ENTIDADES_CIFRADO=true`). **Nunca se envía al LLM**
`eventos_seguridad`	Intentos de prompt injection: origen (justificación/evidencia), extracto anonimizado, hash, revisado

Derechos ARCOP y administración

Tabla	Descripción
`solicitudes_arcop`	Solicitud del titular: tipo, estado, descripción, plazos, respuesta, fechas
`eventos_solicitud_arcop`	Historial: tipo de evento, estado anterior/nuevo, rol e id del actor, comentario
`usuarios_admin`	Administradores `admin_therion` : email único, hash de contraseña, activo, último acceso

6.2 Modelo entidad-relación

```
erDiagram
      rubro ||--o{ usuario_temporal : "Rubro_id"
      rubro ||--o{ benchmark : "Rubro_id"
      dominio ||--o{ matriz_controles : "dominio_id"
      dominio ||--o{ benchmark : "dominio_id"
      dominio ||--o{ resumen : "dominio_id"
      ruta_formativa ||--o{ catalogo_brechas : "ruta_formativa_id"
      catalogo_brechas ||--o{ matriz_controles : "catalogo_brechas_id_brecha"
      catalogo_brechas ||--o{ respuestas : "Catalogo_brechas_id_brecha"
      usuario_temporal ||--o{ evaluacion : "Usuario_temporal_id"
      usuario_temporal ||--o{ solicitudes_arcop : "Usuario_temporal_id"
      evaluacion ||--o{ solicitudes_arcop : "Evaluacion_id"
      evaluacion ||--o{ respuestas : "Evaluacion_id"
      evaluacion ||--o{ resumen : "Evaluacion_id"
      evaluacion ||--o{ respuestas_filtro : "Evaluacion_id"
      preguntas_filtro ||--o{ respuestas_filtro : "pregunta_id"
      matriz_controles ||--o{ respuestas : "Matriz_controles_id_control"
      respuestas ||--o{ evidencias : "Respuestas_id_respuesta"
      solicitudes_arcop ||--o{ eventos_solicitud_arcop : "Solicitudes_arcop_id_solicitu
  d"
      evaluacion ||--o{ eventos_seguridad : "Evaluacion_id"
      matriz_controles ||--o{ eventos_seguridad : "Matriz_controles_id_control"
      respuestas ||--o{ eventos_seguridad : "Respuestas_id_respuesta"
      evidencias ||--o{ eventos_seguridad : "Evidencias_id_evidencia"
      evaluacion ||--o{ llamadas_llm : "Evaluacion_id"
      respuestas ||--o{ llamadas_llm : "Respuestas_id_respuesta"
      evidencias ||--o{ llamadas_llm : "Evidencias_id_evidencia"
      evaluacion ||--o{ sanitizaciones : "Evaluacion_id"
      respuestas ||--o{ sanitizaciones : "Respuestas_id_respuesta"
      evidencias ||--o{ sanitizaciones : "Evidencias_id_evidencia"
```

`usuarios_admin` no tiene relaciones (el actor de un evento ARCOP se guarda como `actor_rol` + `actor_id`).

6.3 Migraciones (Alembic)

Cadena lineal; la cabeza actual es `f7b2d1a8c3e9`.

#	Revisión	Contenido
1	`84d29cadfb28`	Esquema inicial: `evaluacion`, `matriz_controles`, `respuestas`, `evidencias`
2	`20674abc0fc2`	`dominio`, `rubro`, `ruta_formativa`, `usuario_temporal`, `catalogo_brechas`, `benchmark`, `resumen`
3	`b54cdc747a36`	Amplía `matriz_controles` y `catalogo_brechas`
4	`2e398c9a9b2e`	**Seed:** dominios, rubros, rutas, 40 brechas y 40 controles
5	`24ce495af093`	`preguntas_filtro`, `respuestas_filtro`, estado del filtro en `evaluacion`
6	`2d93373ba237`	**Seed:** preguntas F1–F7
7	`7c1e5a9d3f20`	`eventos_seguridad`
8	`a53fb924d0ac`	Ajustes en `eventos_seguridad` y `resumen`
9	`c4f2a8e1b7d3`	Capa de IA: `llamadas_llm`, `sanitizaciones`, revisión humana en `respuestas`
10	`b25cc4eed4b0`	Vacía (sin cambios)
11	`a168283ff2b8`	Clasificaciones a `VARCHAR(25)` y ajustes de columnas de IA
12	`e5a1c9d4b2f7`	`solicitudes_arcop`, `eventos_solicitud_arcop`
13	`f7b2d1a8c3e9`	`usuarios_admin`, plazos ARCOP, `usuario_temporal.fecha_anonimizacion`

Alembic toma la URL de `DATABASE_URL_SYNC` (`migrations/env.py`).

---

7. Endpoints de la API

Base: `http://localhost:8000` (o `http://localhost:8080` a través de nginx). Documentación interactiva en `/docs`. Columna **Auth**: 🔓 público · ☐ cualquier token válido · ☐ titular (`referente` o `encuestado`) · 🛡 `admin_therion`.

Sistema

Método	Ruta	Auth	Descripción
GET	`/health`	🔓	Health check (`{"status":"ok"}`)
GET	`/docs`	🔓	Swagger UI (botón *Authorize* para el token)

Autenticación — `/api/auth`

Método	Ruta	Auth	Body / Respuesta
POST	`/api/auth/login`	🔓	`{email, password}` → token de admin
POST	`/api/auth/titular`	🔓	`{usuario_temporal_id, email}` → token de referente
POST	`/api/auth/encuestado`	🔓	`{evaluacion_id, clave}` → token de encuestado
POST	`/api/auth/encuestado/clave`	🔒	`{evaluacion_id}` → 201 `{clave}` (se muestra una vez; invalida la anterior)
GET	`/api/auth/me`	☐	`{usuario_id, rol}`

Catálogos y controles

Método	Ruta	Auth	Descripción
GET	`/api/dominios`	🔓	4 dominios
GET	`/api/rubros`	🔓	8 rubros
GET	`/api/rutas-formativas`	🔓	12 rutas
GET	`/api/brechas`	🔓	40 brechas con su ruta anidada
GET	`/api/controles?dominio_id=&criticidad=`	🔓	40 controles con dominio y brecha anidados (filtros opcionales; `criticidad` = `Alta` / `Media` / `Baja`)
GET	`/api/controles/{id}`	🔓	Detalle de un control

Evaluaciones, filtro y reportes — `/api/evaluaciones`

Método	Ruta	Auth	Descripción
GET	`/api/evaluaciones?page=&size=&estado_progreso=`	🔓	Lista paginada
GET	`/api/evaluaciones/{id}`	🔓	Detalle
GET	`/api/evaluaciones/{id}/respuestas?page=&size=&estado_final=&fuente=`	🔓	Respuestas paginadas (filtra por clasificación final o fuente)
GET	`/api/evaluaciones/{id}/filtro`	🔓	Preguntas F1–F7, respuestas guardadas y cantidad de controles aplicables
POST	`/api/evaluaciones/{id}/filtro`	🔓	`{"respuestas":[{"codigo":"F1","respuesta":true}, …]}` (las 7) → controles aplicables
GET	`/api/evaluaciones/{id}/controles-aplicables`	🔓	Controles que aplican (400 si el filtro no está completo). El catálogo sale de una caché en memoria (TTL 5 min)
GET	`/api/evaluaciones/{id}/progreso`	🔓	Progreso del cuestionario para reanudar: `etapa` (`filtro` / `cuestionario` / `completo` / `cerrada`), respondidos / aplicables, `por_dimension`, `siguiente_control`, `pendientes`, `respondidos`
POST	`/api/evaluaciones/{id}/cerrar`	🔓	Genera resumen por dominio, puntaje y nivel general (vista previa: **no** cierra la evaluación, se puede repetir)
POST	`/api/evaluaciones/{id}/completar`	🔓	**Cierre definitivo:** exige filtro completo, todos los controles aplicables respondidos y ninguna evidencia en validación; genera el resumen y marca `estado_progreso = completada` (entra al benchmark). 409 `CUESTIONARIO_INCOMPLETO`, `EVIDENCIAS_EN_PROCESO`, `REVISION_SIN_EVIDENCIA` (control en `requiere_revision` sin evidencia), `FILTRO_PENDIENTE` o `EVALUACION_CERRADA`; 404 si no existe
GET	`/api/evaluaciones/{id}/resumen`	🔓	Resumen guardado (404 si no se ha cerrado)
GET	`/api/evaluaciones/{id}/comparativa`	🔓	Resumen vs. benchmark del rubro
GET	`/api/evaluaciones/benchmar`	🔓	Benchmark por rubro × dominio

Método	Ruta	Auth	Descripción
	`k?rubro_id=`		

Paginación: `page` ≥ 1, `size` 1–100; respuesta `{items, total, page, size, pages}`.

Respuestas y evidencias

Método	Ruta	Auth	Descripción
POST	`/api/respuestas/`	🔓	`{evaluacion_id, matriz_control_id, estado_clasificacion, justificacion}` (1 de 5 estados) → **upsert**: 201 si crea, 200 si el control ya estaba respondido; cuerpo `{id_respuesta, creada, actualizada}`. 400 estado inválido, 404 evaluación/control inexistente, 409 evaluación cerrada o filtro pendiente, 422 `JUSTIFICACION_OBLIGATORIA`, `CONTROL_NO_APLICABLE` o `PROMPT_INJECTION`
PUT	`/api/respuestas/{id}`	🔓	`{estado_clasificacion, justificacion}` — modifica una respuesta existente (mismas validaciones)
GET	`/api/respuestas/{id}`	🔓	Respuesta con clasificación final, fuente, confianza y evidencias
PATCH	`/api/respuestas/{id}/revision`	🔓	`{clasificacion_revisada}` — revisión humana; acepta `cumplido`, `parcialmente_cumplido`, `no_cumplido` o `no_aplica` (no `requiere_revision`). Asigna la brecha si queda `no_cumplido` y la quita en otro caso
POST	`/api/respuestas/{id}/evidencias`	🔓	`multipart/form-data` campo `file` → 202 (procesamiento asíncrono); 400 si la respuesta no es `cumplido`/`parcialmente_cumplido` o el archivo está vacío; 409 `EVALUACION_CERRADA` si la evaluación está completada/cerrada
GET	`/api/evidencias/{id}`	🔓	Estado de validación y resultado del LLM

Capa de IA y seguridad

Método	Ruta	Auth	Descripción
GET	`/api/ia/metricas?desde=&hasta=&tarea=&modelo=`	🔓	`operacion_modelo` (tasa de error global y por tipo, latencia p50/p95, tokens, costo), `precision_modelo` (LLM vs. revisión humana: accuracy, P/R/F1, matriz de confusión) y `sanitizacion` (entidades por tipo, fallos, truncados)
GET	`/api/seguridad/eventos?solo_pendientes=&limit=`	🔓	Bandeja de intentos de prompt injection
PATCH	`/api/seguridad/eventos/{id}/revisado`	🔓	Marca un evento como revisado

Derechos ARCOP — `/api/arcop`

Método	Ruta	Auth	Descripción
GET	`/api/arcop/catalogo`	🔓	Tipos, estados, transiciones y plazos configurados
POST	`/api/arcop/solicitudes`	☐	`{tipo, descripcion}` → 201, estado `recibida`
GET	`/api/arcop/seguimiento/{codigo}`	🔓	Estado y plazos por código de seguimiento (sin datos personales)
GET	`/api/arcop/solicitudes?page=&size=&estado=&tipo=&usuario_temporal_id=&evaluacion_id=&vencidas=`	☐	Titular: solo las suyas. Admin: todas, con filtros
GET	`/api/arcop/solicitudes/{id}`	☐	Detalle e historial (404 si es de otro titular)
PATCH	`/api/arcop/solicitudes/{id}/estado`	🔒	`{estado, comentario?}` → `en_revision` / `pendiente_informacion` / `en_proceso`
POST	`/api/arcop/solicitudes/{id}/prorroga`	🔒	`{motivo}` → prórroga única del plazo (+30 días), antes de que venza
POST	`/api/arcop/solicitudes/{id}/resolucion`	🔒	`{resultado: resuelta|rechazada, respuesta}`
POST	`/api/arcop/solicitudes/{id}/cancelacion`	☐	`{motivo?}`
POST	`/api/arcop/solicitudes/{id}/informacion`	☐	`{informacion}` → vuelve a `en_revision`

Códigos de error

HTTP	Código (`detail.codigo`)	Cuándo
400	— (texto)	Validación de negocio
401	`NO_AUTENTICADO`, `TOKEN_INVALIDO`, `CREDENCIALES_INVALIDAS`, `USUARIO_DESHABILITADO`	Autenticación
403	`NO_AUTORIZADO`	Rol insuficiente
404	—	Recurso no encontrado
409	`TRANSICION_INVALIDA` (incluye `estado_actual` y `permitidas`)	Cambio de estado ARCOP no permitido
422	`PROMPT_INJECTION`	Justificación bloqueada
503	`AUTH_NO_CONFIGURADA`	Falta `AUTH_SECRET`
503	`ANONIMIZACION_NO_DISPONIBLE`	HU-06: el motor NER falló; no se guardó la justificación ni la evidencia
503	`CIFRADO_NO_CONFIGURADO`	HU-06: falta `ENTITY_MAP_KEY` para cifrar evidencias

---

8. Capa de Políticas e IA

Intermediario entre la lógica de negocio y el LLM (`app/application/ai/`). Los casos de uso solo piden una **tarea** (`TareaIA.VALIDAR_EVIDENCIA`); la capa se encarga del resto.

```
caso de uso ─► AIGateway.ejecutar(tarea, textos, datos, contexto)
    1. Router        PoliticaTarea: prompt versionado, schema, límites, campos no conf
  iables
    2. Sanitización  motor NER sobre cada texto no confiable ── falla ⇒ ABORTA (no lla
  ma al LLM)
    3. Auditoría     mapa de entidades CIFRADO en `sanitizaciones` ── no se puede guar
  dar ⇒ ABORTA
    4. Prompt        plantilla versionada (validar_evidencia/1.2.0) SOLO con texto san
  itizado
    5. Proveedor     LLMProviderPort → GroqProvider (reintentos 429/5xx, timeout)
    6. Validación    schema Pydantic + reglas post-salida (p. ej. divergencia sin expl
  icar)
    7. Métricas      una fila en `llamadas_llm` por ejecución, con éxito o con tipo de
  error
```

Reglas de negocio: R1 el texto original nunca llega al LLM · R2 el mapa de entidades se guarda en BD y no va al proveedor · R3 si la sanitización falla se aborta y se registra · R4 si el LLM falla se registra `exito=False` con el tipo de error (`sanitizacion`, `auditoria`, `timeout`, `rate_limit`, `autenticacion`, `conexion`, `proveedor`, `salida_invalida`, `desconocido`) · R5 etiquetas fijas por tipo · R6 `SANITIZER_ENABLED=false` solo para depurar (prohibido con `APP_ENV=production`).

Salida esperada del LLM (`EvidenceValidation`): `clasificacion` (`cumplido` | `parcialmente_cumplido` | `no_cumplido`), `confianza` (0–1), `justificacion`, `respalda_clasificacion_usuario`, `divergencia_explicada`, `evidencia_suficiente`, `datos_extraidos`, `errores`.

Modelo por defecto: `openai/gpt-oss-safeguard-20b` vía Groq (debe soportar *structured outputs*), temperatura 0.

Motor NER (`infrastructure/security/ner_sanitizer.py`, versión 1.1.0): híbrido reglas + spaCy `es_core_news_md`. Detecta primero todos los spans, fusiona solapados y reemplaza en una sola pasada, lo que permite devolver el mapa (tipo, valor, inicio, fin) para auditoría. Etiquetas: `[RUT_ANON]` (con validación de dígito verificador), `[NOMBRE_ANON]`, `[CORREO_ANON]` (incluye ofuscaciones tipo "arroba"/"punto"), `[TEL_ANON]`, `[DIRECCION_ANON]`, `[DATO_CODIFICADO_ANON]` (Base64/Hex/Binario/Base32 hasta 3 niveles). El worker no arranca si falta el modelo de spaCy.

Rendimiento (T-04, < 2,0 s por página): el motor procesa una página de 3.000 caracteres en **25 ms (p95)** dentro del Docker del proyecto. Para eso: (1) carga solo el componente `ner` del modelo (mismas entidades, ~45 % menos de CPU); (2) la segunda pasada para nombres en MAYÚSCULAS solo revisa ventanas alrededor de nombres de 2 o más palabras, no el documento entero; (3) un único modelo por proceso, compartido con `seguridad.py`; (4) el modelo se precarga al iniciar el worker y la API; (5) la anonimización corre en un hilo (`asyncio.to_thread`) para no bloquear el event loop. En precisión, las referencias normativas ("Ley 21.719", "artículo 14") ya no se toman como direcciones, los cargos ("Presidenta del Directorio") se recortan de los nombres, y se anonimizan correos con tildes, "cargo + nombre" marcados como organización y direcciones tipo "Los Leones 456". Se evaluó ONNX y no se adoptó: el cuello de botella no era la inferencia. Detalle y mediciones en `backend/CAMBIOS_T04.md`.

Cifrado: Fernet con `ENTITY_MAP_KEY`; las entidades además llevan una huella HMAC que permite buscar coincidencias sin descifrar. Guarda la clave fuera del repositorio: sin ella los mapas ya guardados no se pueden descifrar.

Agregar una tarea nueva (p. ej. recomendación de ruta formativa): crear su prompt en `ai/prompts/<tarea>.py` con `PROMPT_VERSION`, su schema Pydantic y registrar una `PoliticaTarea` en `crear_politicas`. **Cambiar de proveedor:** crear un adaptador de `LLMProviderPort` en `infrastructure/llm/`, registrarlo en `factory.py` y cambiar `LLM_PROVIDER` / `LLM_MODEL`.

---

9. Seguridad y privacidad (Ley 21.719)

Riesgo	Control implementado	Dónde
Datos personales enviados a un tercero (LLM)	Anonimización NER obligatoria, fail-closed; recorte después de sanitizar para no cortar PII a medias	`ner_sanitizer.py`, `gateway.py`
Re-identificación desde la auditoría	Mapa de entidades sin valores originales (solo huellas HMAC y conteos, HU-06); métricas sin texto del usuario	`cifrado.py`, `registro_ia_repository.py`
PII guardada en BD, disco o logs (HU-06)	Justificación y nombre de archivo anonimizados antes de persistir (503 si el motor falla); evidencias cifradas en reposo y reemplazadas por su texto anonimizado tras validar; nombres de archivo en disco = UUID; textos ARCOP cifrados (excepción); prueba de regresión que siembra PII	`anonimizador.py`, `archivo_cifrado_storage.py`, `cifrado_columnas.py`, `test_hu06_regresion_pii.py`
Prompt injection en justificaciones	Detector por reglas con normalización (URL-decode, NFKC, homoglifos, invisibles, leetspeak, datos codificados); bloqueo 422 y registro	`seguridad.py`, `prompt_guard.py`, `create_respuesta.py`
Prompt injection en evidencias	`MODO_SEGURIDAD_EVIDENCIA=bloquear` (no se llama al LLM, respuesta en `requiere_revision`) o `marcar` (se registra y no hay fallback al usuario)	`process_evidence.py`
Que una inyección suba el puntaje	`requiere_revision` vale 0 y cuenta en el denominador	`generate_resumen.py`
Derechos del titular	Solicitudes ARCOP con trazabilidad completa y anonimización irreversible al aprobar una supresión	`gestionar_solicitud_arcop.py`, `anonimizacion_repository.py`
Credenciales	PBKDF2-SHA256 600k, tokens HMAC con expiración, revocación inmediata, errores genéricos	`autenticacion.py`
Configuración insegura en producción	La app no arranca con `APP_ENV=production` si `SANITIZER_ENABLED=false`, falta `ENTITY_MAP_KEY` o `AUTH_SECRET` < 32 caracteres	`config.py`
Exposición de servicios	Puertos publicados solo en `127.0.0.1`; contenedor con usuario sin privilegios	`docker-compose.yml`, `Dockerfile`

> `infrastructure/security/seguridad.py` *es una copia del repositorio separado* ***ComplianceAI-Prompt-Injection****. Si se actualiza allá (nuevos patrones o tests), hay que volver a copiarlo aquí.*

10. Frontend

Estado: demo estática navegable, servida por nginx en `http://localhost:8080`. **Todavía no consume el backend**: `js/store.js` implementa una "API" simulada (Promises sobre `localStorage`) con la misma forma que tendrá el cliente real basado en `fetch`, y `js/data.js` contiene la matriz, el árbol de aplicabilidad, módulos formativos, evaluaciones precargadas y el panel sectorial de ejemplo.

Cuentas de la demo (cualquier contraseña): `nuevo@therionlabs.cl` (referente sin diagnóstico), `completo@therionlabs.cl` (referente con diagnóstico cerrado), `admin@therionlabs.cl` (`admin_therion`).

Página	Función	Rol
`index.html`	Entrada / redirección según sesión	—
`login.html`	Ingreso	—
`dashboard.html`	Panel principal: estado del diagnóstico y brechas prioritarias	ambos
`diagnostico.html`	Inicio del autodiagnóstico	referente
`cuestionario.html`	Preguntas de compuerta y de control, carga de evidencias	referente
`resumen.html`	Diagnóstico cerrado: resumen de madurez	referente
`informe.html`	Informe de madurez por dimensión y brechas priorizadas; exporta PDF (jsPDF)	referente
`formacion-mapa.html`	Mapa formativo por dimensión y nivel	referente
`formacion-modulo.html`	Contenido de un módulo formativo	referente
`formacion-progreso.html`	Progreso de formación	referente
`panel-sectorial.html`	Benchmark por rubro	ambos
`panel-metodologia.html`	Metodología de evaluación	ambos
`admin-matriz.html`	Backoffice: edición de la matriz de controles	admin_therion
`admin-arbol.html`	Backoffice: árbol de aplicabilidad	admin_therion

Diferencias con el backend que hay que resolver al integrar (ver deuda #3): estados abreviados (`parcial`, `no`, `na`, `revision`), `revision` pondera 50 en la demo y 0 en el backend, y la escala de niveles de la demo (*Riesgo bajo/moderado/alto*, *Adecuado/Parcial*) no coincide con la del backend (*inicial* … *optimizado*).

11. Datos maestros

Dominios: 1 Protección de Datos · 2 Gobernanza de IA · 3 Gestión de Riesgos · 4 Uso Responsable.

Rubros: 1 Banca y Finanzas · 2 Salud · 3 Retail y E-commerce · 4 Telecomunicaciones · 5 Educación · 6 Tecnología/SaaS · 7 Manufactura · 8 Servicios/Otros.

Rutas formativas (12): cada dominio × {Inicial, Intermedio, Avanzado}. Cada brecha apunta a una ruta (no necesariamente de su mismo dominio; p. ej. `ERR-DAT-002` → *Gestión de Riesgos – Intermedio*).

Controles (40):

Prefijo	Dominio	Marco principal	Alta	Media	Baja
DAT-001…010	Protección de Datos	Ley N.º 21.719	7	3	0
GOB-001…010	Gobernanza de IA	ISO/IEC 42001	3	5	2
RIE-001…010	Gestión de Riesgos	NIST AI RMF 100-1 (RIE-004: ISO/IEC 42001)	5	5	0
SEG-001…010	Uso Responsable	OWASP LLM, NIST 600-1 (SEG-006/007), Ley 21.719 (SEG-008/010)	8	1	1
Total			**23**	**14**	**3**

Cada control tiene su brecha `ERR-<prefijo>-<n>` (40 en total). La fuente de verdad es la migración `2e398c9a9b2e_seed_catalogs_and_controls.py`.

---

12. Puesta en marcha

12.1 Con Docker (recomendado)

Solo se necesita **Docker Desktop** (en Windows, con WSL 2). No hace falta Python ni spaCy en la máquina.

1. Crear `backend/.env` desde la plantilla y completar las claves:

```
cd Proyecto
  copy backend.env.example backend.env
```

Variable	Cómo obtenerla
`GROQ_API_KEY`	console.groq.com/keys
`ENTITY_MAP_KEY`	`python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"`
`AUTH_SECRET`	`python -c "import secrets; print(secrets.token_urlsafe(48))"` (necesaria para `/api/auth` y `/api/arcop`)

Sin Python local: `docker run --rm python:3.13-slim sh -c "pip install -q cryptography && python -c 'from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())'"`.

2. Levantar todo:

```
docker compose up -d --build
```

La primera vez tarda varios minutos (descarga imágenes e instala spaCy con su modelo).

URL	Qué es
http://localhost:8080	Frontend (demo)
http://localhost:8000/docs	Swagger de la API
`localhost:5433`	PostgreSQL (`postgres` / `1234`, BD `complaince_db`) — solo desarrollo
`localhost:6379`	Redis

3. Crear un administrador (opcional, para ARCOP):

```
docker compose exec api python -m app.core.crear_admin --email admin@therionlabs.cl
```

En Docker se ignoran `DATABASE_URL`, `DATABASE_URL_SYNC`, `REDIS_URL` y `STORAGE_DIR` del `.env` (el compose los reemplaza por los servicios internos). Si cambias el `.env`, aplica con `docker compose up -d`.

12.2 Manual (para depurar la API o el worker)

Requisitos: Python 3.12+ (probado en 3.13), Docker para PostgreSQL y Redis.

```
cd Proyecto
  docker compose up -d db redis

  cd backend
  python -m venv venv
  .\venv\Scripts\Activate.ps1          # si PowerShell lo bloquea: Set-ExecutionPolicy
  -ExecutionPolicy RemoteSigned -Scope CurrentUser
  pip install --upgrade pip
  pip install -r requirements.txt      # incluye spaCy y el wheel de es_core_news_md 3.
  8.0
  copy env.example .env                # valores para 127.0.0.1:5433 / complaince_db; a
  grega AUTH_SECRET
  alembic upgrade head

  # Terminal 1
  uvicorn app.main:app --reload --port 8000
  # Terminal 2
  arq app.worker.tasks.WorkerSettings
```

Notas: usa `127.0.0.1` y no `localhost` (en Windows `localhost` resuelve primero a IPv6). Si `python -m spacy download es_core_news_md` falla por incompatibilidad con pydantic, el wheel ya viene fijado en `requirements.txt`.

12.3 Preparar datos de prueba

Mientras no existan endpoints de alta (deuda #1), se insertan por SQL:

```
docker compose exec db psql -U postgres -d complaince_db -c "INSERT INTO usuario_tempor
  al (fecha_expiracion, email, validacion_invitacion, `"Rubro_id`") VALUES ('2027-12-3
  1', 'test@example.com', 'S', 6);"
  docker compose exec db psql -U postgres -d complaince_db -c "INSERT INTO evaluacion
  (fecha_inicio, estado_progreso, `"Usuario_temporal_id`", filtro_completado) VALUES (C
  URRENT_DATE, 'en_progreso', 1, false);"
```

`filtro_completado` es `NOT NULL` sin valor por defecto en la BD, por eso se indica explícitamente. En PowerShell las comillas dobles internas se escapan con acento grave (`` ` ``).

12.4 Recorrido de prueba (PowerShell)

```
$api = "http://localhost:8000/api"

  # 1. Filtro: usa IA y LLM (F1 y F4 = SI)
  Invoke-RestMethod -Method Post "$api/evaluaciones/1/filtro" -ContentType "applicatio
  n/json" -Body '{"respuestas":[{"codigo":"F1","respuesta":true},{"codigo":"F2","respue
  sta":false},{"codigo":"F3","respuesta":false},{"codigo":"F4","respuesta":true},{"codi
  go":"F5","respuesta":false},{"codigo":"F6","respuesta":false},{"codigo":"F7","respues
  ta":false}]}'

  # 2. No cumplido -> brecha automática
  Invoke-RestMethod -Method Post "$api/respuestas/" -ContentType "application/json" -Bo
  dy '{"evaluacion_id":1,"matriz_control_id":3,"estado_clasificacion":"no_cumplido","ju
  stificacion":"No tenemos politica formal."}'

  # 3. Cumplido + evidencia (procesada por el worker)
  $r = Invoke-RestMethod -Method Post "$api/respuestas/" -ContentType "application/jso
  n" -Body '{"evaluacion_id":1,"matriz_control_id":1,"estado_clasificacion":"cumplid
  o","justificacion":"Politica aprobada."}'
  curl.exe -X POST "$api/respuestas/$($r.id_respuesta)/evidencias" -F "file=@backend/ev
  idencia.txt"
  Start-Sleep 10
  Invoke-RestMethod "$api/respuestas/$($r.id_respuesta)" | ConvertTo-Json -Depth 5

  # 4. Evidencia maliciosa -> bloqueo preventivo y evento de seguridad
  curl.exe -X POST "$api/respuestas/$($r.id_respuesta)/evidencias" -F "file=@backend/ev
  idencia_maliciosa.txt"
  Invoke-RestMethod "$api/seguridad/eventos"

  # 5. Cerrar y ver reporte
  Invoke-RestMethod -Method Post "$api/evaluaciones/1/cerrar"
  Invoke-RestMethod "$api/evaluaciones/1/comparativa"
  Invoke-RestMethod "$api/ia/metricas"
```

El recorrido ARCOP (login, titular, solicitudes) está en `backend/CAMBIOS_ARCOP.md`.

---

13. Variables de entorno

Variable	Defecto	Descripción
`DATABASE_URL`	— (obligatoria)	URL async (`postgresql+asyncpg://…`)
`DATABASE_URL_SYNC`	— (obligatoria)	URL síncrona para Alembic (`postgresql+psycopg2://…`)
`REDIS_URL`	— (obligatoria)	Redis para ARQ
`GROQ_API_KEY`	— (obligatoria)	Clave de Groq
`LLM_PROVIDER`	`groq`	Proveedor del LLM
`LLM_MODEL`	`openai/gpt-oss-safeguard-20b`	Modelo (con *structured outputs*)
`LLM_TIMEOUT_S` / `LLM_MAX_REINTENTOS`	`60` / `2`	Timeout por intento y reintentos (429/5xx)
`LLM_MAX_TOKENS_SALIDA`	`1500`	Tope de tokens de respuesta
`LLM_PRECIO_ENTRADA_USD_1M` / `LLM_PRECIO_SALIDA_USD_1M`	`0` / `0`	Precio del modelo para estimar costo en métricas
`UMBRAL_CONFIANZA`	`0.8`	Confianza mínima para aceptar la clasificación del LLM
`JUSTIFICACION_MIN_CARACTERES`	`20`	HU-02: largo mínimo de la justificación en `parcialmente_cumplido` y `requiere_revision`
`MAX_CHARS_EVIDENCIA_LLM` / `MAX_CHARS_JUSTIFICACION_LLM`	`8000` / `4000`	Caracteres enviados al LLM (tras sanitizar)
`MODO_SEGURIDAD_EVIDENCIA`	`bloquear`	`bloquear` o `marcar` ante posible inyección en evidencias
`SANITIZER_ENABLED`	`true`	Anonimización NER antes del LLM (`false` solo debug)
`ENTITY_MAP_KEY`	—	Clave maestra Fernet: huellas del mapa, cifrado de evidencias y de textos ARCOP (obligatoria)
`GUARDAR_MAPA_ENTIDADES_CIFRADO`	`false`	HU-06: `true` reactiva el mapa reversible (debe documentarse como excepción)
`REQUIERE_NER`	`false`	**Obsoleta** (reemplazada por `SANITIZER_ENABLED`)
`APP_ENV`	`development`	`development` · `test` · `production` (activa validaciones estrictas)
`STORAGE_DIR`	`./storage`	Carpeta de evidencias (`/data/storage` en Docker)
`AUTH_SECRET`	—	Secreto HMAC de tokens (≥ 32 caracteres en producción)

Variable	Defecto	Descripción
`AUTH_TOKEN_TTL_MIN`	`480`	Vigencia del token
`ARCOP_PLAZO_RESPUESTA_DIAS`	vacío	Plazo de respuesta ARCOP (días corridos). **Dejar comentada** hasta definirla: con `=` vacío la app no arranca
`ARCOP_PLAZO_INFORMACION_DIAS`	vacío	Plazo para aportar información (mismo cuidado)
`POSTGRES_USER` / `POSTGRES_PASSWORD` / `POSTGRES_DB`	`postgres` / `1234` / `complaince_db`	Solo `docker-compose.yml` (variables del shell o `Proyecto/.env`)

---

14. Pruebas y calidad

Las pruebas unitarias e integración **no necesitan PostgreSQL, Redis ni Groq**: usan repositorios falsos o SQLite en memoria (`aiosqlite`).

Archivo	Pruebas	Qué cubre
`test_bordes_na_y_evidencia.py`	42	Escenarios borde: N/A masivo, falta de evidencia, cálculo de puntaje, una respuesta por control, evidencias en evaluación cerrada
`test_hu01_cuestionario.py`	14	HU-01: upsert y PUT de respuestas, rechazo de controles no aplicables y evaluaciones cerradas, `controles-aplicables` optimizado, progreso (la de carrera de 10 POST corre solo live)
`test_completar_evaluacion.py`	9	`POST /completar`: condiciones, cierre atómico y bloqueo de respuestas, filtro y evidencias
`test_hu02_matriz_estados.py`	24	HU-02: 5 estados, justificación obligatoria (unitarias y API), `requiere_revision` resuelta por el LLM, revisión humana, requisito de evidencia al completar y brecha coherente con el estado vigente
`test_solicitudes_arcop.py`	24	ARCOP: ciclo de vida, roles, login, anonimización, plazos
`test_ner_sanitizer.py`	18	Motor NER, cifrado del mapa y precisión contra `tests/data/ner_gold.jsonl` (17 casos)
`test_rendimiento_ner.py`	14	T-04: < 2,0 s por página, pipeline mínimo, segunda pasada acotada, NER fuera del event loop y precisión ≥ 0,95 / cobertura 100 % sobre 40 páginas etiquetadas
`test_capa_ia.py`	17	AIGateway contra las reglas R1–R6
`test_seguridad_integracion.py`	11	Capa anti prompt-injection en el backend
`test_metricas_ia.py`	6	Cálculo de métricas (funciones puras)
`test_e2e_capa_ia.py`	2	Caso end-to-end del sistema con la capa de IA
`test_latencia_api.py`	1	Latencia de la API REST: p95 < 500 ms por endpoint (en proceso o live)
`test_defectos_api.py`	1	Variante anterior de la prueba de latencia (ver deuda #12)
`test_hu06_anonimizacion.py`	20	HU-06: etiquetas fijas, fail-closed, archivo cifrado y reemplazado por el texto anonimizado, mapa irreversible, textos ARCOP cifrados
`test_hu06_regresion_pii.py`	1	HU-06 C3: siembra PII por todas las entradas y verifica que no aparece en ninguna tabla (lectura en crudo), en disco, en los logs ni en lo enviado al LLM
`test_live_capa_ia.py`	1	Sistema real: uvicorn + ARQ + PostgreSQL + Redis + Groq + spaCy (se activa con `LIVE_BASE_URL`)

Soporte: `conftest.py` (variables mínimas y storage temporal), `_api_harness.py` (siembra datos de prueba), `_capa_ia_fakes.py` (dobles de prueba), `_corpus_ner.py` (corpus sintético etiquetado para

medir el NER).

```
# Suite completa en Docker (no toca la BD real)
  docker compose run --rm --no-deps api pytest tests -q

  # Suite local (Windows, venv en backend/)
  pytest tests -v -rxX

  # Precisión del NER con detalle
  pytest tests/test_ner_sanitizer.py -s

  # Rendimiento del NER (T-04): pruebas y benchmark
  docker compose exec worker pytest tests/test_rendimiento_ner.py -v -s
  docker compose exec worker python scripts/benchmark_ner.py --paginas 100 --evidencia
  8000
  docker compose exec worker python scripts/benchmark_ner.py --paginas 20 --perfil   #
  + profiling

  # Prueba en vivo contra el sistema levantado (usa la BD real, evaluación 9101)
  docker compose exec api sh -c 'LIVE_BASE_URL=http://localhost:8000 LIVE_DATABASE_URL=
  $DATABASE_URL LIVE_ENTITY_MAP_KEY=$ENTITY_MAP_KEY pytest tests/test_live_capa_ia.py -
  v -s'
```

Últimos resultados reportados

- **T-04 · Motor NER (15-oct-2026, Docker, 8 núcleos):** página de 3.000 caracteres en 25,1 ms (p95; límite 2.000 ms), evidencia de 8.000 caracteres en 66,6 ms (p95). Precisión 0,988 y cobertura de PII 100 % sobre 40 páginas etiquetadas. Suite completa: 212 aprobadas, 1 omitida (prueba en vivo).
- **Prueba en vivo (13-oct-2026):** PASSED con `openai/gpt-oss-safeguard-20b` y spaCy `es_core_news_md` 3.8.0. Sin fugas de PII, mapa cifrado correcto e inyección bloqueada sin llamar al LLM. Falsos positivos del NER observados: "la Ley 21" como dirección y "Presidenta del Directorio" como nombre (**corregidos en T-04**, motor v1.1.0).
- **Latencia live (11-oct-2026, concurrencia 10):** 12 de 13 endpoints cumplen p95 < 500 ms. No cumplía `GET controles-aplicables` (p95 542,7 ms). Detalle en `backend/reporte_latencia_live_c10.md`.
- **HU-01 (15-oct-2026):** `controles-aplicables` optimizado (1 consulta + catálogo cacheado). **Latencia live oficial (16-oct-2026, Docker, concurrencia 10): 12/12 endpoints cumplen**; `GET controles-aplicables` p95 24,1 ms (antes 542,7 ms). Detalle en `backend/reporte_latencia_live_hu01.md` y `backend/CAMBIOS_HU01.md`.

---

15. Comandos útiles y solución de problemas

```
docker compose ps                          # estado de cada servicio
  docker compose logs -f worker            # procesamiento de evidencias
  docker compose logs migrate              # si la API no arranca, mirar aquí primero
  docker compose up -d --build api worker  # tras cambiar código del backend
  docker compose down                      # detener (conserva datos)
  docker compose down -v                   # detener y BORRAR BD y evidencias

  docker compose exec db psql -U postgres -d complaince_db           # consola SQL
  docker compose exec db psql -U postgres -d complaince_db -c "\dt"  # listar tablas
  docker compose exec redis redis-cli ping                           # PONG

  docker compose exec api alembic current                            # revisión aplicad
  a
  docker compose exec api alembic history --verbose
  alembic revision --autogenerate -m "descripcion"                   # nueva migración
  (modo manual)
```

Problema	Causa probable	Solución
Puerto 5433, 6379, 8000 u 8080 en uso	Servicios locales fuera de Docker	Detener el uvicorn/worker/Redis/PostgreSQL local
`worker` se reinicia en bucle	Falta `ENTITY_MAP_KEY` o `GROQ_API_KEY`, o el modelo de spaCy	`docker compose logs worker`
`migrate` falla	Error de migración o BD no disponible	`docker compose logs migrate`
503 `AUTH_NO_CONFIGURADA`	Falta `AUTH_SECRET`	Agregarla al `.env` y `docker compose up -d`
La app no arranca tras definir plazos ARCOP	Variable con `=` vacío	Comentarla o poner un número
`ConnectionDoesNotExistError` (asyncpg)	`localhost` resuelve a IPv6	Usar `127.0.0.1`
`404 model does not exist` (Groq)	Modelo deprecado	`LLM_MODEL=openai/gpt-oss-safeguard-20b`
`ModuleNotFoundError: No module named 'app'`	Carpeta equivocada	Ejecutar desde `backend/`
`column "Matriz_controles_id_control" does not exist`	PostgreSQL pasa a minúsculas	Envolver en comillas dobles
`-Form` no se reconoce	PowerShell 5.1	Usar `curl.exe -F "file=@archivo"`
Caracteres raros (`polÃtica`)	Codificación de la consola	Verificar en `psql`; suele ser solo visual

---

16. Backlog, deuda técnica y riesgos

Priorizado para la planificación del próximo sprint. Los números se usan como referencia en el resto del documento.

Prioridad alta

#	Ítem	Impacto	Propuesta
1	**No hay endpoints para crear invitaciones (`usuario_temporal`) ni evaluaciones**	El flujo completo depende de SQL manual	`POST /api/invitaciones` (admin) con código aleatorio y `POST /api/evaluaciones` (referente)
2	**Solo `/api/arcop` y `/api/auth/me` exigen token**: respuestas, evidencias, revisión, métricas, eventos de seguridad y reportes son públicos y no verifican que la evaluación pertenezca al usuario	Exposición de datos entre organizaciones	Aplicar `requerir_rol` y control de propiedad en todos los routers
3	**Frontend no integrado** con el backend (`store.js` simulado) y con reglas distintas (`revision` 50 vs 0; escalas de nivel distintas)	La demo no refleja los resultados reales	Cliente `fetch` con la misma interfaz que `store.js`; unificar puntaje y niveles en una sola fuente
4	~~Benchmark siempre vacío: ningún endpoint fijaba `completada`~~ **Resuelto:** `POST /completar` marca la evaluación como `completada`	—	Pendiente decidir si se permite reabrir una evaluación completada
5	~~Falta `backend/.dockerignore`~~ **Resuelto (15-oct):** `backend/download` renombrado a `backend/.dockerignore`; el `.env` y el `venv` ya no entran en la imagen	—	—

Prioridad media

#	Ítem	Propuesta
6	Las evidencias se leen como texto UTF-8: PDF, DOCX e imágenes llegan al LLM como texto ilegible. La API no limita tamaño (solo nginx, 20 MB) ni tipo MIME	Extracción de texto por tipo (pypdf, python-docx, OCR opcional), lista blanca de MIME y tope de tamaño en la API
7	~~`GET /evaluaciones/{id}/controles-aplicables` p95 542 ms~~ **Resuelto en HU-01** (consulta única + caché del catálogo): p95 24,1 ms en la medición live del 16-oct	—
8	NER: los falsos positivos "la Ley 21" y "Presidenta del Directorio" se corrigieron en T-04. Queda abierto: textos **completamente en mayúsculas** con frases largas, donde spaCy puede no aislar el nombre	Regla adicional para texto 100 % en mayúsculas; ampliar `ner_gold.jsonl` con casos reales anonimizados
9	`LLM_PRECIO_*` en 0: el costo estimado sale siempre 0	Cargar precios del modelo en el `.env`
10	Decisiones ARCOP abiertas: plazos y qué pasa al vencer, código de invitación, límite de intentos de login, verificación de identidad, duplicados	Definir con el PO; detalle en `CAMBIOS_ARCOP.md`
11	Sin CRUD de administración de matriz y árbol en el backend (solo en la demo) ni versionado de controles	Endpoints admin + migración de versionado

Prioridad baja

#	Ítem
12	Limpieza: `.env.example` y `env.example` desalineados (el segundo no tiene `AUTH_*`); `test_defectos_api.py` parece una versión anterior de `test_latencia_api.py`; `domain/ports/document_validator_port.py` importa un módulo inexistente (`schemas.validation`) y no se usa; migración vacía `b25cc4eed4b0`
13	`seguridad.py` se sincroniza a mano con el repo ComplianceAI-Prompt-Injection: riesgo de divergencia (evaluar paquete compartido o submódulo). Además, su regex de correo no acepta tildes (`matías.díaz@…` queda en parte visible en los extractos de eventos de seguridad): corregirla allá, como se hizo en `ner_sanitizer.py`
14	Fechas de respuestas/evidencias en `Date` (sin hora): limita la trazabilidad fina
15	Credenciales por defecto de PostgreSQL (`postgres` / `1234`) en el compose: aceptables solo en desarrollo
16	Pipeline CI/CD (pytest + build de imagen) y entorno de staging (Sprint 4)

---

17. Documentación complementaria

Documento	Contenido
`backend/CAMBIOS_ARCOP.md`	Historia de usuario ARCOP: archivos, acceso, anonimización, plazos, endpoints y decisiones pendientes
`backend/reporte_latencia_live_c10.md`	Último reporte de latencia de la API
`backend/CAMBIOS_HU06.md`	HU-06: anonimización antes de persistir, decisiones del PO, excepción ARCOP y migración de datos existentes
`backend/CAMBIOS_T04.md`	T-04: optimización del motor NER (cambios, mediciones antes/después, decisión sobre ONNX)
`../Documentos/Fase 1/Presentación Proyecto.pdf`	Presentación: cliente, problema, solución, equipo, sprints, TRL
`../Documentos/Fase 1/Bitácora CPCI TT 2025-2.pdf`	Bitácora del proyecto
`../Documentos/Fase 1/`, `../Documentos/Fase 2/`	Guía del estudiante, evaluaciones formativas, autoevaluaciones y diarios de reflexión por integrante
`../Extras/Guia_ComplianceAI_PromptInjection.docx.pdf`	Guía del módulo anti prompt-injection

Licencia: proyecto académico (Capstone APT122, Duoc UC) desarrollado para Therion Labs.